

Everything the dashboard does, and how to use it

Zoffec Tally Ledger mirrors your Tally company in a live dashboard: who owes you, who you owe, what's in stock, and whether bank payments actually landed — updated automatically as you work in Tally, no manual exports. This guide walks every screen and setting, in the order you'll actually touch them.

GETTING STARTED

Running it, and pulling updates

Two ways to run it, and one way to keep it current after a change like this one.

Recommended — runs at every login

1. Open a terminal in the `backend` folder.
2. Run `powershell -ExecutionPolicy Bypass -File install_task.ps1`.
3. That registers a Windows Task Scheduler job named **TallyTracker** that starts the dashboard hidden (no console window) at logon, restarts it automatically if it ever crashes, and builds the frontend as part of the install.

The dashboard is then always at <http://127.0.0.1:8731/> from the next logon onward — it also starts immediately the first time. It sits quietly whenever Tally isn't open and starts syncing on its own the moment you open Tally again.

TASK	COMMAND
Check status	<code>Get-ScheduledTask -TaskName TallyTracker Get-ScheduledTaskInfo</code>
Stop it	<code>Stop-ScheduledTask -TaskName TallyTracker</code>
Remove entirely	<code>powershell -ExecutionPolicy Bypass -File uninstall_task.ps1</code> (from <code>backend</code>)

Manual — foreground, for one-off runs

```
cd backend
python -m uvicorn main:app --host 127.0.0.1 --port 8731
```

Pulling an update

Whenever the code changes (like this hardening pass), bring it in with:

```
git pull
cd backend
python -m pip install -r requirements.txt
cd ../frontend
npm install
npm run build
```

Then restart the server (or the scheduled task) so it picks up the new code. Any schema change applies itself automatically on that next start — see Backups & data safety.

Tip If you only installed the scheduled task, `npm run build` already happens as part of `install_task.ps1` — re-running that script after a pull covers both steps at once.

GETTING STARTED

Dashboard tour

The sidebar on the left is the whole app — six sections, always visible, collapsible if you want more table width. Click the `Collapse` control at the bottom to shrink it to icons only; it remembers your preference.

The top strip

Four tiles sit above every section: **Total Receivables**, **Total Payables**, **Net Position** (receivables minus payables), and **Overdue Entities** (how many customers/vendors currently have at least one overdue bill). These update live along with everything else.

Next to it, the header shows sync status — "*Synced 2 min ago · live*" when things are current, or an amber offline banner reading "*Tally offline — showing data as of ...*" when Tally itself isn't reachable. The dashboard keeps showing your last known data either way; it never goes blank just because Tally is closed.

Refresh now, top-right, forces an immediate sync instead of waiting for the next automatic poll (every 5 minutes by default — see Configuration).

Entities

Every customer and vendor ledger from Tally, in one searchable table.

- **Filter** — the All / Customers / Vendors pill switch above the table.
- **Search** — matches ledger name as you type (debounced, so it doesn't refetch on every keystroke).
- **Sort** — click any column header (Name, Type, Balance, Open Bills, Overdue Bills, Last Changed); click again to reverse direction. The active column shows an arrow.
- **Paginate** — 50 rows per page, Prev/Next controls at the bottom.
- **Export CSV** — top-right button; exports the *full* filtered/sorted result, not just the visible page.
- **Open a row** — click anywhere on it to open that entity's detail panel. Rows tinted red have at least one overdue bill.

WORKING WITH DATA

Entity detail

Opens as an overlay panel from Entities, or from a row in Payable/Receivable. Press Esc or click outside it to close.

Bills

Every bill for that entity — ref, bill date, due date, outstanding amount, status (open / overdue / closed). Overdue rows are tinted red.

Transactions

Live vouchers pulled directly from Tally at the moment you open the panel — these are **not** stored in the dashboard's own database, by design, so this list can't go stale or grow without bound. If Tally is closed when you open a panel, this table shows *"Tally offline — can't load transactions right now"* instead of stale data.

Balance history

A line chart of this entity's balance across every sync snapshot the dashboard has taken. Needs a few syncs of history before it has anything to plot.

Followups

Your own running notes on this entity — completely separate from Tally sync, never touched or overwritten by it. Type a note, pick a status (**Pending** / **Contacted** / **Resolved**), and add it. Newest note shows first.

WORKING WITH DATA

Stock

Every stock item from Tally — quantity, unit, value. Same sort/paginate/export pattern as Entities.

- **Low stock only** — checkbox above the table; filters to items below the configured threshold (see [Configuration](#)). Rows below threshold are tinted regardless of the filter, so you can spot them even browsing the full list.

WORKING WITH DATA

Payable & Receivable

Payable is every open bill you owe a vendor. **Receivable** is every open bill a customer owes you. Same table shape both ways, sorted soonest-due-first.

Reading the status badge

14d overdue

Due in 3d

Due in 22d

No due date

BADGE	MEANING
Overdue	Due date already passed — always shown regardless of amount.
Due soon	Due within the next 7 days.
Later	Due in more than 7 days.
No due date	Extremely rare — see the fallback rule below.

How due dates are worked out Tally doesn't always carry an explicit due date on a bill. When it's missing — or when Tally's own data would imply a 0-day credit period, which usually means the field isn't really set — the dashboard falls back to **bill date + your configured credit days** (30 by default) instead of leaving the bill undated at the bottom of the list.

Summary line

Above the table: total bill count, total outstanding amount across *all* matching bills (not just the visible page), and how many are currently overdue.

Sort, paginate, export CSV, and click-to-open-entity all work exactly as in Entities.

WORKING WITH DATA

Bank reconciliation

Upload a bank statement; the dashboard suggests which open bill each transaction pays, based on amount, payer name, and date — you confirm or correct every suggestion. Nothing here ever writes back to Tally or changes a bill's status; it's purely local tracking of what's been reconciled.

1. Upload

1. Export your bank statement as `.xlsx` from your bank's portal — any standard export works, column names are auto-detected (Date/Value Date/Txn Date, Narration/Description/Particulars, Debit/Credit or Withdrawal/Deposit amounts).
2. Drag it onto the upload zone, or click to browse.
3. Previously uploaded statements stay available in the dropdown at top-right, so you can switch back between them.

If upload fails You'll see "*Could not process that file*" — this means the parser couldn't find a header row with recognizable Date/Narration/Debit-Credit columns in the first 15 rows. Check your export includes a proper header row and standard column names.

2. Review suggested matches

Each transaction row shows a confidence bar and the suggested entity + bill:

CONFIDENCE	WHAT IT MEANS
85–100	Strong match — amount and payer name both line up closely. Still requires your confirmation, never auto-applies.
40–84	Possible match — worth a look before confirming, especially if the name score is doing most of the work.
below 40	Not shown at all — row displays "no match found" and needs a manual pick.

The score weighs amount most heavily (50%), payer name (40%, fuzzy-matched after stripping bank noise like NEFT/UTR codes), and date proximity to the bill's due date (10%, neutral if either date is missing).

3. Act on each row

- **Confirm** — accepts the suggested match as-is.
- **Change / Pick manually** — opens a search box; type an entity name, pick it from the typeahead list, then pick the specific bill from the dropdown, then **Confirm this**.

- **Ignore** — for transactions that aren't a bill payment at all (bank charges, internal transfers, etc.) — marks it resolved without linking it to anything.

The line below the table — "*N of M still need review*" — tracks how much of the statement is left to work through.

WORKING WITH DATA

Activity feed

A running log of what changed on each sync — balance changes, new bills appearing, bills being settled, stock quantity changes. Useful as a quick "what happened since I last looked" check without hunting through individual entities.

KEEPING IT HEALTHY

Live updates & health

The dashboard holds a live connection to the server (Server-Sent Events) and refetches whatever you're looking at the moment new data lands — that's the "· *live*" tag next to the sync status. If that connection drops for any reason, it falls back to polling every 30 seconds instead, so it degrades gracefully rather than going stale silently.

Behind that, a background sync runs on its own timer (`polling.interval_minutes` in config, 5 minutes by default) independent of anyone having the dashboard open at all — **Refresh now** and the timer both call the exact same sync path.

Health check

`GET /api/health` — a plain JSON status check, separate from whether Tally itself is reachable:

```
{
  "status": "ok",
  "database_ok": true,
  "tally_reachable": true,
  "last_synced_at": "2026-08-13T09:41:02+05:30"
}
```

Useful if you ever want to point an external uptime check at the app, or just to confirm the server process itself is alive versus Tally being the thing that's down.

KEEPING IT HEALTHY

Backups & data safety

Every poll cycle checks whether today already has a backup; if not, it copies the database into `backend/backups/` — so you get one automatic snapshot per calendar day with zero setup, and the last 14 are kept.

A backup is also taken automatically right before any future schema update runs against your existing database, so a code update can never leave you without a fallback copy if something about the update goes wrong.

Restoring from a backup

1. Stop the server (or `Stop-ScheduledTask -TaskName TallyTracker`).
2. In `backend/backups/`, find the snapshot you want — filenames are timestamped, e.g. `tally-daily-20260813-000754.db.bak`.
3. Copy it over `backend/tally.db`.
4. Restart the server.

Note Restoring a backup rolls back *everything* in the dashboard's own database — followups, reconciliation history, sync history — to that point in time. It has no effect on Tally itself; the next sync repopulates entities/bills/stock from Tally as normal.

Configuration

Everything lives in `backend/config.yaml`. Restart the server after any change.

KEY	DEFAULT	WHAT IT CONTROLS
<code>tally.host</code>	<code>localhost</code>	Where Tally's HTTP-XML server is listening.
<code>tally.port</code>	<code>9000</code>	Tally's configured TCP/IP port (Gateway of Tally → F1 → Settings → Connectivity).
<code>tally.company_name</code>	—	Pins requests to one company by name, so results don't silently depend on whichever company happens to be open. Required for multi-company Tally installs.
<code>polling.interval_minutes</code>	<code>5</code>	How often the background sync runs on its own.
<code>stock.low_stock_threshold</code>	<code>10</code>	Quantity below which an item is flagged low-stock.
<code>billing.default_credit_days</code>	<code>30</code>	Fallback due-date window when Tally gives no usable due-date signal — see Payable & Receivable.
<code>database.path</code>	<code>./tally.db</code>	Where the SQLite file lives.

Troubleshooting

"Tally offline" even though Tally is open

- Gateway of Tally → F1 (Help) → Settings → Connectivity → Client/Server configuration — confirm the HTTP-XML server is enabled and the port matches `tally.port` in config.
- A company must actually be loaded in Tally, not just the app running.
- Run `python backend/tools/phase0_raw_probe.py` with Tally open — it hits the raw endpoint and prints exactly what came back, or exactly why it couldn't connect.

Bills or balances look wrong for your Tally version

Run `python backend/verify_live_tally.py` with Tally open. If a value comes back empty or wrong, re-run with `TALLY_CLIENT_DEBUG=1` set to see Tally's actual raw XML — the tag lookups live entirely in `backend/tally_client.py`, the one file that knows Tally's XML shape.

Recent bills missing from Payable/Receivable

Check what's actually in the database before assuming it's a display bug:

```
cd backend
python -c "import sqlite3; conn = sqlite3.connect('tally.db'); [print(r) for r in conn.execute(\"SELECT subs
```

If a year is genuinely absent, it's usually Tally's own "Current Period" setting (Gateway of Tally → F2/Alter Period) limiting what a sync can see — the dashboard requests a wide explicit date range specifically to avoid this, but a very old install predates that fix. `git pull` and restart if you haven't recently.

Followup or reconciliation action fails silently

A toast in the bottom corner reports every action's result — success or failure, in plain language (e.g. *"Couldn't confirm that match — try again"*). If nothing appears at all, check the server is actually reachable at `127.0.0.1:8731`.